Spring Security + JWT Authentication Flow (Detailed Guide)

This document explains the authentication and authorization flow in a Spring Boot application using Spring Security and JWT. Each component is explained with its responsibility and how it participates in the overall request flow. This is written for beginners.

1. Why Do We Need Authentication & Authorization?

Authentication answers the question: WHO are you? Authorization answers the question: WHAT are you allowed to do? Spring Security helps us implement both in a standardized and secure way.

2. Stateless Authentication (JWT Concept)

In a stateless system, the server does not store login information. Each request must contain proof that the user is authenticated. JWT (JSON Web Token) is that proof.

- Server does NOT create HTTP sessions
- Server does NOT remember logged-in users
- Client sends JWT on every protected request

3. Core Components and Their Responsibilities

SecurityConfig

Defines security rules, builds the filter chain, and decides which URLs need authentication.

JwtFilter

Runs on every request, validates JWT, and sets authentication in SecurityContext.

AuthenticationManager

Main entry point that performs username & password authentication during login.

AuthenticationProvider

Contains the logic for how authentication is done (DB, LDAP, etc.).

UserDetailsService

Loads user details (username, password, roles) from the database.

JwtService

Generates JWT tokens and validates them.

SecurityContextHolder

Stores authentication information for the current request.

4. Login Flow (Username & Password Authentication)

Login happens once. It uses Spring Security's AuthenticationManager to validate credentials.

```
Client
 |
 | POST /auth/login
 | (username + password)
 v
AuthController
 |
 | Calls authenticationManager.authenticate()
 v
AuthenticationManager
 |
 v
DaoAuthenticationProvider
 |
 | Calls UserDetailsService
 v
UserDetailsService
 |
 | Fetch user from DB
 v
PasswordEncoder (BCrypt)
 |
 | Compare passwords
 v
Authentication SUCCESS
 |
 v
JwtService.generateToken()
 |
 v
JWT returned to client
```

5. JWT Token – What It Contains and Why

JWT is a signed token that contains user identity information. It allows the server to trust the request without storing session data.

- Subject (username / email)
- Issued time (iat)
- Expiration time (exp)
- Signature (ensures token is not tampered)

6. Protected API Request Flow (JWT Validation)

Every protected API request must include the JWT in the Authorization header.

```
Client
 |
 | GET /api/hello
 | Authorization: Bearer <JWT>
 v
Security Filter Chain
```

```
 |
 v
JwtFilter
 |
 | Extract token from header
 | Validate signature and expiration
 | Load user from DB
 | Create Authentication object
 | Set SecurityContextHolder
 |
 v
Authorization Check
 |
 v
Controller
```

7. What Happens If JWT Is Missing or Invalid?

If a protected API is accessed without a valid JWT, Spring Security blocks the request before it reaches the controller.

8. Role of SecurityContextHolder

SecurityContextHolder stores authentication data for the current request thread. Spring Security checks this context to decide whether a request is authenticated.

9. End-to-End Mental Model

```
REGISTER:
Client -> Controller -> Save User

LOGIN:
Client -> Controller
       -> AuthenticationManager
       -> AuthenticationProvider
       -> UserDetailsService
       -> PasswordEncoder
       -> JWT returned

EVERY PROTECTED REQUEST:
Client -> JwtFilter
       -> Validate JWT
       -> SecurityContextHolder
       -> Authorization
       -> Controller
```

10. Final Takeaway

Spring Security does not magically authenticate users. Authentication happens only when filters or code explicitly set the SecurityContext. JWT + JwtFilter provide authentication in a stateless system.